

[프로젝트 제안서] KMA-Live : 기상청 API 기반 날씨 정보 웹

3조 팀원 : 이승규 박진현 강호현 최하람

1. 프로젝트 개요

1.1. 프로젝트명

KMA-Live: 기상청 API 기반 실시간 날씨 정보 웹사이트

1.2. 프로젝트 목표

대한민국 기상청에서 제공하는 공공데이터 API를 활용하여, 지정된 지역(서울)의 현재 날씨 정보와 상세 기상 변수 및 한반도 위성 영상을 사용자에게 시각적으로 제공하는 것을 목표로 한다.

Python의 Flask 프레임워크를 사용하여 API 데이터를 웹 페이지에 표시하는 과정을 보인다.

1.3. 개발 환경 및 사용 기술

프로그래밍 언어 : Python

웹 프레임워크 : Flask

프론트엔드 : HTML

API : 대한민국 기상청 공공데이터포털

2. 요구사항 명세

2.1. 제공 기능

현재 날씨 정보 조회: 서울 지역의 현재 기온, 시간당 강수량, 풍속, 하늘 상태(맑음, 구름 많음, 흐림 등) 정보를 텍스트 형태로 제공한다. 추가적으로, 체감 온도, 습도, 풍향 등 상세 기상 변수도 함께 제공한다.

다중 뷰 내비게이션: 상단 메뉴바를 통해 '홈(현재 날씨)', 상세 예보, 레이더 지도'의 3가지 뷰를 클릭하여 전환할 수 있도록 구현한다.

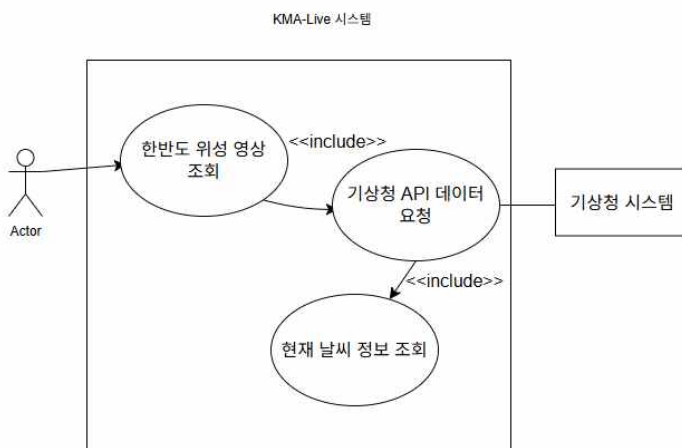
시간별 상세 예보: 향후 시간대별 기온, 강수 확률, 풍속 등의 데이터를 표 형태로 제공하는 전용 뷰를 구현한다.

한반도 위성 영상 조회: HSR(Hybrid Surface Rainfall) 레이더 합성 영상 이미지를 전용 뷰에 표시한다. (구현상 실시간으로 데이터를 받아오는 데 어려움이 있어 1시간 정도 이상으로 시간 기준은 길어질 수 있다.)

2.2. 사용자 시나리오

1. 사용자가 웹 브라우저를 통해 KMA-Live 웹사이트에 접속한다.
2. 웹사이트 메인 페이지가 로딩되면서, 시스템은 자동으로 기상청 서버에 서울의 최신 날씨 데이터를 요청한다.
3. 동시에, 시스템은 기상청 서버에 가장 최신의 한반도 위성 영상 이미지를 요청한다.
4. 사용자는 별도의 조작 없이 메인 페이지에서 '서울 현재 날씨' 카드와 '한반도 위성 영상' 카드를 통해 날씨 정보를 한눈에 확인할 수 있다.

2.3. UML Use Case 다이어그램



현재 날씨 정보 조회: 사용자가 웹사이트에서 서울의 날씨 정보를 본다.

한반도 위성 영상 조회: 사용자가 웹사이트에서 한반도 위성 영상을 본다.

(내부) 기상청 API 데이터 요청: 위 기능들을 수행하기 위해 시스템이 내부적으로 기상청 서버와 통신한다.

3. 개발 전략

3.1. 역할 분담

- 백엔드 담당 (2명)

Python Flask 서버 환경 설정 및 실행

기상청 API 명세 분석 및 데이터 요청/응답 처리 로직 구현

서버와 프론트엔드 간의 데이터 전달 로직 구현

- 프론트엔드 담당 (2명)

HTML을 이용한 3가지 뷰 웹 페이지 구조 설계

Bootstrap을 활용한 UI/UX 디자인 및 스타일링

백엔드로부터 전달받은 데이터를 화면에 표시하는 템플릿 코드 작성

3.2. 개발 일정 (총 4주)

- 1주차 (기획 및 환경설정)
프로젝트 목표 및 기능 정의, 제안서 작성
기상청 API 활용 신청 및 서비스 키 발급
Python, Flask 개발 환경 설정
- 2주차 (백엔드 개발)
기상청 단기에보 API 호출 및 데이터 파싱 기능 구현
3가지 뷰(홈, 상세 예보, 레이더 지도) 라우팅 설정 및 백엔드 데이터 연동
기상청 위성 영상 URL 생성 로직 구현
- 3주차 (프론트엔드 개발):
메인 페이지 HTML 구조 설계
Bootstrap CSS를 적용하여 화면 레이아웃 및 디자인 구현
메뉴바를 통한 뷰 전환 기능의 구현
- 4주차 (통합 및 테스트)
백엔드와 프론트엔드 코드 통합
데이터가 정상적으로 표시되는지 테스트 및 오류 수정
최종 발표 자료 준비

4. 위험 요소 및 평가 방법

4.1. 위험 요소 및 해결 방안

위험 요소 1 : 기상청 API 응답 지연 또는 실패

내용 : 기상청 서버의 상태나 네트워크 문제로 API 응답이 없거나 오류가 발생할 수 있다.

해결 방안 : 코드 내에 try-except 구문을 사용하여 예외 처리를 하고, 데이터 로딩 실패 시 사용자에게 "정보를 불러오는 데 실패했습니다."와 같은 메시지를 명확하게 표시한다.

위험 요소 2: API 호출 규격 변경

내용 : 기상청에서 API의 주소나 요청/응답 데이터 형식을 변경할 수 있다.

해결 방안 : 프로젝트 진행 중 문제가 발생하면, 가장 먼저 기상청 공공데이터포털의 API 문서를 확인하여 변경 사항이 있는지 대조한다.

4.2. 평가 방법

정상 동작 여부 : 웹사이트 접속 시, 서울 날씨 정보와 위성 영상이 오류 없이 화면에 출력.

3가지 뷰(홈, 상세 예보, 레이더 지도)가 메뉴 클릭에 따라 정상적으로 전환되는지 확인.

데이터 정확성 : 웹사이트에 표시되는 날씨 정보가 실제 기상청 홈페이지의 정보와 일치

코드 완성도 : 주석과 변수명을 통해 각 코드의 역할을 명확히 설명

5. 참고문헌 및 용어 설명

5.1. 참고문헌

기상청 공공데이터포털 : <https://data.kma.go.kr/>

=> 현재 공공데이터포털 서비스 장애로 인해 대체 사이트로 프로젝트 수행



대체 사이트 : <https://apihub.kma.go.kr/>

Flask 공식 문서 : <https://flask.palletsprojects.com/>

Bootstrap 공식 문서 : <https://getbootstrap.com/>

5.2. 용어 설명

API (Application Programming Interface): 프로그램들이 서로 소통하기 위한 약속. 이 프로젝트에서는 우리 프로그램이 기상청 서버의 데이터를 가져오기 위해 사용하는 통신 규칙

Flask : 파이썬으로 웹 서버를 쉽고 빠르게 만들 수 있게 도와주는 도구 (웹 프레임워크)

격자 좌표 (NX, NY): 기상청에서 대한민국 지도를 작은 사각형(격자)으로 나누어 각 위치에 부여한 고유한 X, Y 좌표. 특정 지역의 날씨를 정확히 알기 위해 사용된다.

템플릿 엔진 (Jinja2): HTML 문서 안에 데이터가 들어갈 자리를 표시({{ }})해두면, 파이썬에서 전달한 실제 데이터를 그 자리에 채워 넣어 동적인 웹 페이지를 만들어주는 기술.

HSR (Hybrid Surface Rainfall): 지형 차폐의 영향이 없는 지상에 가장 가까운 고도각 자료로 추정하여 레이더 강우량의 정확도를 높인 레이더 강우량 자료.